

# DepGraph-Guided Pruning for Noise Mitigation in NoisyGCN

313554040黃念暉 313554056鄭翔祐 312554056紀品榕

## Introduction

1) Robustness Defined by NoisyGCN: NoisyGCN introduces an  $\varepsilon$ - $\gamma$  adversarial risk framework, demonstrating that training with Gaussian noise of scale  $\beta$  provides an upper bound  $\gamma$  on this risk.

- If  $f$  is GCN-based, then  $f$  is  $(\varepsilon, \gamma)$ -robust with

$$\gamma = \frac{2(\|W^{(2)}\| \|W^{(1)}\| \|X\| \varepsilon)^2}{\beta},$$

- If  $f$  is GIN-based, then  $f$  is  $(\varepsilon, \gamma)$ -robust with

$$\gamma = \frac{2(\|W^{(2)}\| \|W^{(1)}\| \|X\| \varepsilon (2\|A\| + \varepsilon))^2}{2\beta},$$

where  $W(\ell)$  is the weight matrix of the  $\ell$ -th layer.

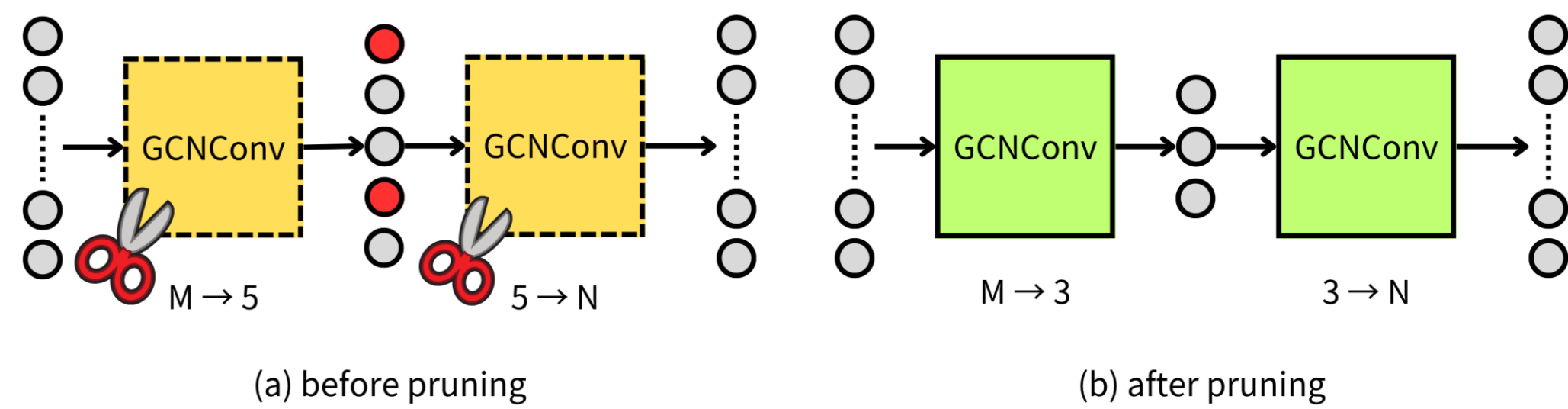
2) Moderate sparsification reduces inference costs while maintaining or even enhancing accuracy against structural poisoning and adversarial attacks.

## Objective

We aim to enhance the robustness of GNNs against noise by integrating noise injection, guided by the  $\gamma$  upper bound formula proposed in the NoisyGNN paper (as shown above). This formula serves as the theoretical foundation for our method, where we attempt to minimize  $\gamma$  by increasing  $\beta$  and reducing  $\|W\|$  through pruning.

## Results & Conclusion

## Methodology



$$\mathcal{R}(g, k) = \sum_{k=1}^K \gamma_k \cdot I_{g,k} = \sum_{k=1}^K \sum_{w \in g} \gamma_k \|w[k]\|_2^2 \quad (4)$$

### Algorithm Training and pruning for NoisyGCN

**Input:** graph  $G = (A, X)$ , training iters  $t1$ , finetuning iters  $t2$ , prune ratio  $\tau$ , node class label  $L$

**Output:** model parameter  $\theta$ ;

- 1: initialize the NoisyGCN  $F$  and its parameters  $\theta$ ;
- 2: initialize the Pruner  $P$ ;
- // need  $F$  to build the dependency graph  $DG$  for group pruning;

**Stage1:** training the  $\theta$  for  $t1$  iters

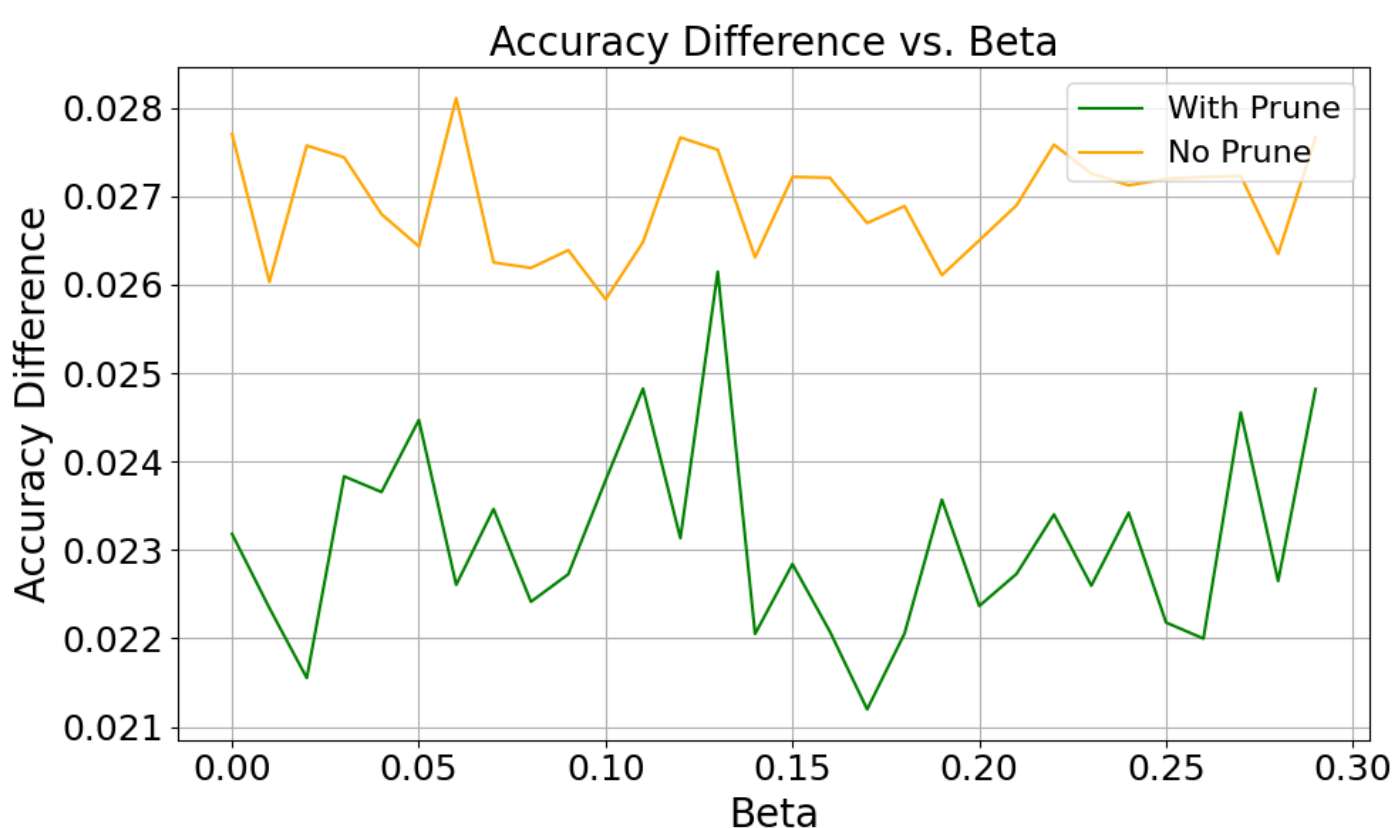
- 3:  $\text{loss} \leftarrow \text{NLL}(F(A, X; \theta), L)$ , backward propagate to get the gradient;
- 4:  $\theta.\text{weight.grad} \leftarrow \theta.\text{weight.grad} + P.\beta \cdot P.\text{regularization}()$ ;
- //  $P.\text{regularization}()$  is Equation (4);
- 5: update  $\theta$  by the gradient;

**Stage2:** pruning

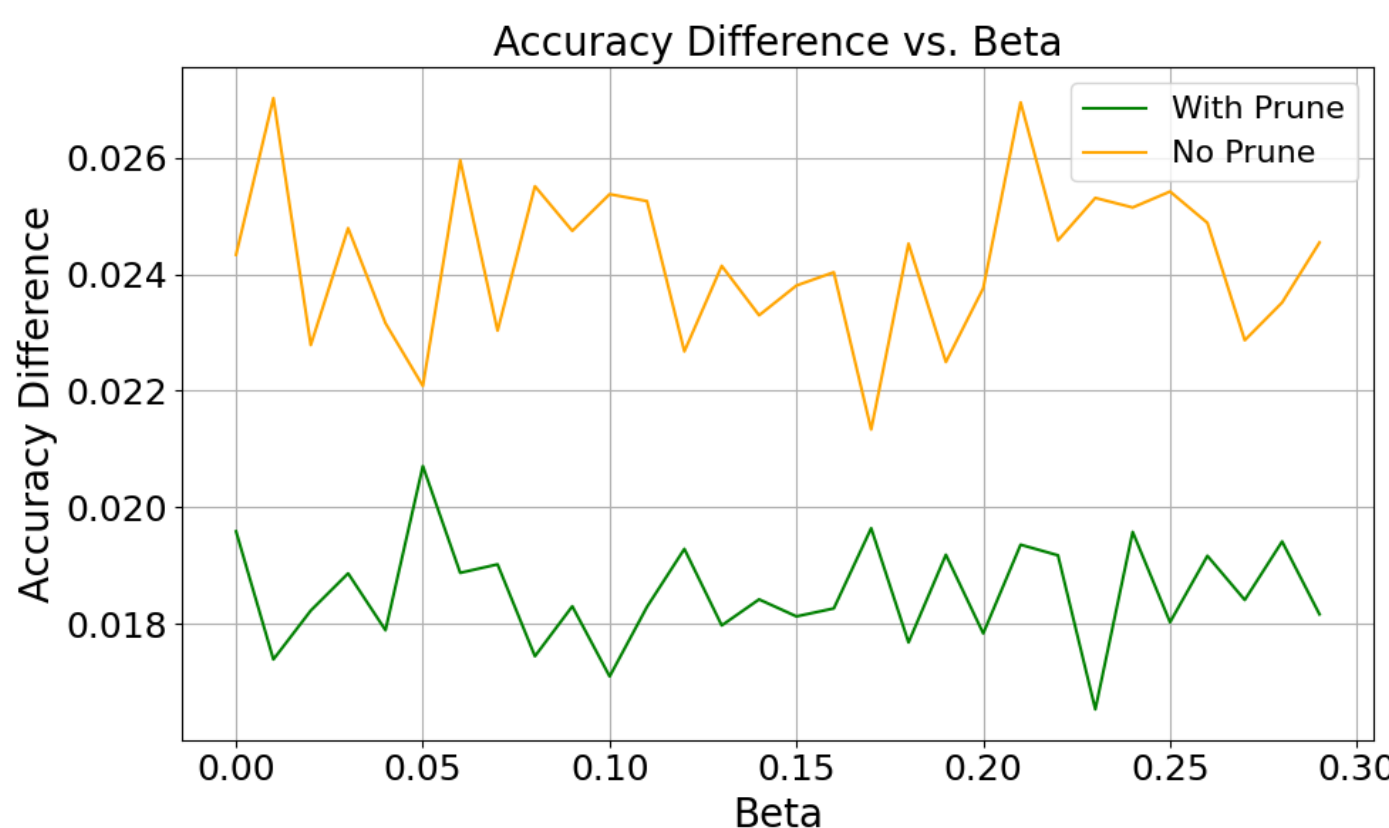
- 6: chose the greatest  $\theta$  from stage1;
- 7:  $\theta' \leftarrow P.\text{prune}(\theta, \tau)$ ;
- // Prune the  $\tau$  of  $\theta$  according to the  $DG$  when initialize  $P$ ;

**Stage3:** fine-tuning the  $\theta'$  for  $t2$  iters

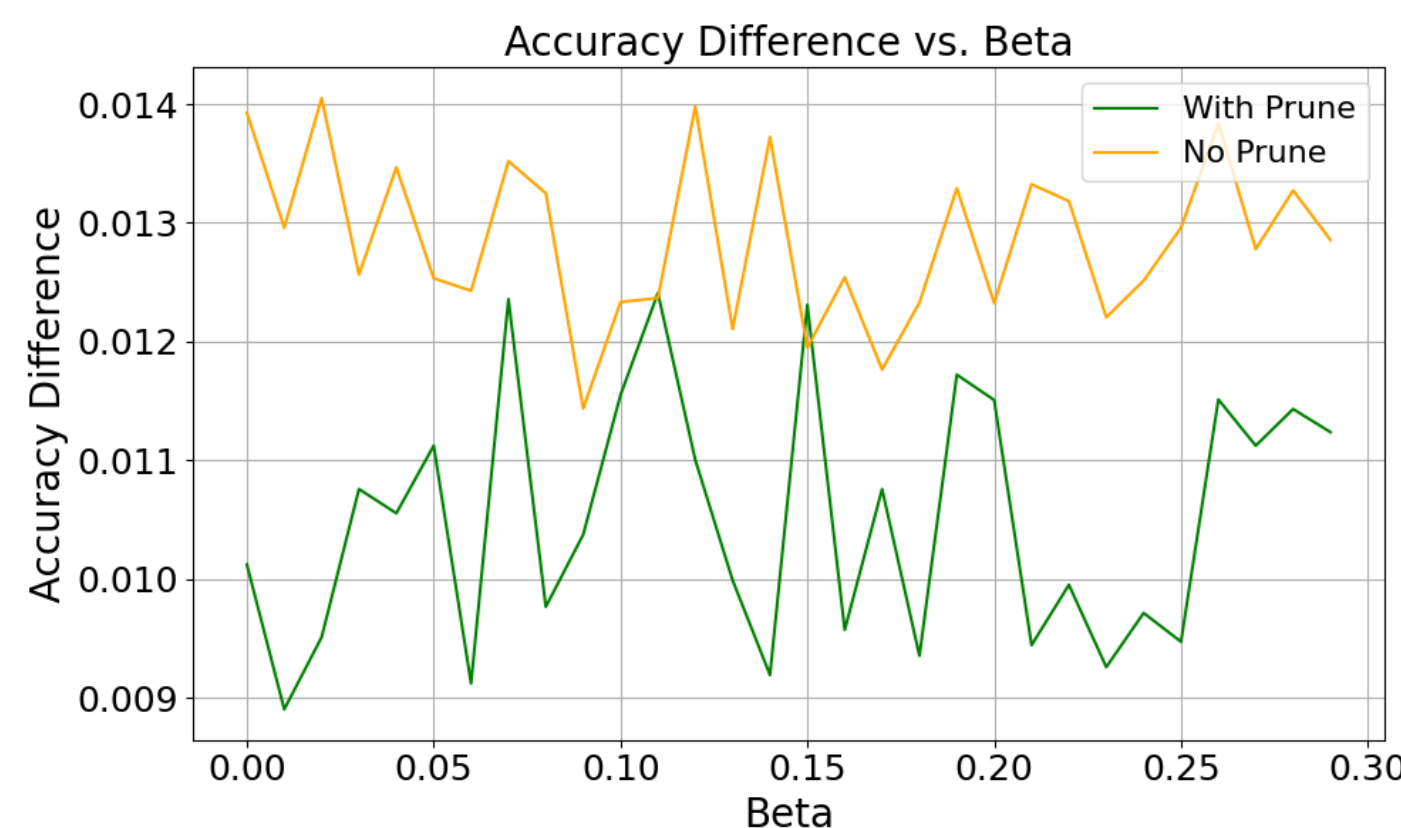
- 8:  $\text{loss} \leftarrow \text{NLL}(F(A, X; \theta'), L)$ , backward propagate to get the gradient;
- 9: update  $\theta'$  by the gradient;
- 10: return  $\theta'$ .



Dice 10% Attack on Cora dataset



Dice 10% Attack on CiteSeer dataset



Dice 5% Attack on Cora dataset

|         | Dataset  | $\epsilon$ | Guard    | Jaccard         | SVD             | RGNN     | Noisy           | Pruning+Noisy   |
|---------|----------|------------|----------|-----------------|-----------------|----------|-----------------|-----------------|
| Mettack | Cora     | 0%         | 77.5±0.7 | 80.9±0.7        | 80.6±0.4        | 83.5±0.3 | 83.2±0.4        | <b>84.3±0.5</b> |
|         |          | 5%         | 75.8±0.6 | 78.9±0.8        | 78.4±0.6        | 78.3±0.6 | <b>81.2±0.7</b> | 79.9±0.9        |
|         |          | 10%        | 74.7±0.4 | <b>76.7±0.7</b> | 71.5±0.8        | 70.7±0.8 | 74.5±0.6        | 74.3±1.1        |
|         | CiteSeer | 0%         | 70.1±1.5 | 71.2±0.7        | 70.7±0.4        | 72.3±0.5 | 71.9±0.4        | <b>74.1±0.6</b> |
|         |          | 5%         | 69.9±1.1 | 70.3±2.3        | 68.9±0.7        | 70.6±0.7 | <b>72.3±0.6</b> | 71.3±0.8        |
|         |          | 10%        | 70.0±1.5 | 67.5±2.1        | 68.8±0.6        | 68.7±1.2 | <b>70.4±0.8</b> | 67.0±1.4        |
|         | PolBlogs | 0%         | 93.1±0.6 | –               | 86.5±0.8        | 94.9±0.3 | <b>95.2±0.4</b> | 94.9±0.3        |
|         |          | 5%         | 72.8±0.8 | –               | <b>85.1±1.6</b> | 76.0±0.8 | 79.7±0.6        | 76.7±1.0        |
|         |          | 10%        | 68.7±1.0 | –               | <b>84.8±2.3</b> | 69.2±1.2 | 73.4±0.5        | 70.8±1.3        |
| DICE    | Cora     | 5%         | 76.4±0.4 | 79.6±0.6        | 74.9±1.3        | 81.9±0.6 | 82.5±0.8        | <b>83.2±0.5</b> |
|         |          | 10%        | 76.6±0.5 | 78.6±0.8        | 73.5±1.5        | 80.0±0.6 | 80.5±0.6        | <b>81.7±0.6</b> |
|         | CiteSeer | 5%         | 68.5±1.6 | 70.9±0.4        | 69.4±1.6        | 69.3±0.5 | 70.8±0.3        | <b>73.0±0.7</b> |
|         |          | 10%        | 69.9±1.5 | 69.9±0.6        | 68.1±1.5        | 67.8±1.1 | 70.4±0.8        | <b>72.1±1.2</b> |
|         | PolBlogs | 5%         | 81.3±0.7 | –               | 86.5±2.3        | 89.6±0.4 | <b>90.3±0.3</b> | 89.9±1.4        |
|         |          | 10%        | 78.9±0.6 | –               | 85.3±2.8        | 85.5±0.9 | 82.1±2.3        | <b>85.7±1.1</b> |

For random attacks like DICE, the combination of Prune and NoisyGNN offers an effective defense that boosts accuracy. However, against targeted attacks like Mettack, it cannot improve accuracy.

Meanwhile, we found that the pruned model indeed reduces the gap between perturbed data and clean data. However,  $\beta$  does not decrease this gap as it increases. We speculate that this is because  $\beta$  affects clean and perturbed data to different degrees.